

יתרונות המעבר מ-Java 8 ל-Java 21

מסמך זה מסביר בשפה פשוטה מה אפשר לעשות ב-Java 21 שהיה קשה או בלתי אפשרי ב-Java 8, באמצעות דוגמאות מתוך **מערכת הבנקאות** שלנו.

כל פרק בנוי לפי אותו מבנה:

- **הבעיה ב-Java 8** – מה היה כואב
- **הפתרון ב-Java 21** – איך זה נעשה קל יותר
- **דוגמה בנקאית** – קוד מתוך הדומיין שלנו
- **למה לבנק להתעניין** – ערך עסקי

1. Virtual Threads – לטפל בהרבה יותר לקוחות במקביל

הבעיה ב-Java 8

כאשר הבנק מעבד תשלומים, העברות או בדיקות הונאה, כל בקשה רצה בדרך כלל ב-**thread** משלה. ב-Java 8, thread הוא יקר: הוא תופס כ-**1 MB של זיכרון**, ומערכת ההפעלה מסוגלת לנהל רק כמה אלפים כאלה במקביל.

לכן, אם רוצים לטפל ב-**10,000 לקוחות** שמעבירים כסף בו-זמנית, אי אפשר פשוט ליצור 10,000 threads. צריך להשתמש ב-**thread pool** (לדוגמה, threads 200), ואת כל השאר להעמיד בתור. כאשר thread מחכה לתשובה ממסד הנתונים, הוא פשוט יושב חסום, לא עושה כלום – אבל עדיין תופס את 1 MB שלו.

זו הסיבה שבנקים על Java 8 צריכים לעיתים להוסיף **שרתים נוספים** ביום שישי השחור או ביום המשכורת: כל שרת מוגבל בכמות הפעולות הבו-זמניות.

הפתרון ב-Java 21

Java 21 מביאה **Virtual Threads**. הם נראים ומתנהגים בדיוק כמו threads רגילים, אבל הם **כמעט חינם**: כ-1 KB במקום 1 MB. אפשר ליצור **מאות אלפים** של כאלה על שרת בודד.

כש-virtual thread מחכה (לתשובה ממסד נתונים, ל-API חיצוני, לרשת), הוא פשוט יורד מהדרך ונותן לאחרים לעבוד. כשהתשובה מגיעה – הוא ממשיך מאותה נקודה.

דוגמה בנקאית

thread pool – Java 8, דורש כיוונון זהיר:

```

ExecutorService executor = Executors.newFixedThreadPool(200);

for (TransferRequest req : todaysSalaryPayments) {
    executor.submit(() -> transactionService.transfer(
        req.fromAccount(), req.toAccount(), req.amount(), "Salary"
    ));
}

```

אם בשעה 9:00 בבוקר מגיעות 50,000 העברות משכורת, 49,800 מהן יושבות בתור.

Java 21 – virtual thread אחד לכל תשלום:

```

try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (TransferRequest req : todaysSalaryPayments) {
        executor.submit(() -> transactionService.transfer(
            req.fromAccount(), req.toAccount(), req.amount(), "Salary"
        ));
    }
}

```

כל 50,000 התשלומים רצים בו-זמנית. על אותו שרת בדיוק.

למה לבנק להתעניין

- פחות עלויות חומרה. אותה מכונה משרתת פי 10 עד פי 100 יותר לקוחות.
- חוויית לקוח טובה יותר. אין יותר "אגא המתן" בשעות העומס.
- קוד פשוט יותר. מפתחים כותבים לוגיקה ישירה מלמעלה למטה, במקום שרשראות callback מסובכות.

2. Records – פחות קוד, פחות באגים

הבעיה ב-Java 8

בנק עובד עם הרבה אובייקטים קטנים: `Receipt`, `CustomerSummary`, `Address`, `Transaction`. ב-Java 8, כל מחלקה כזו דורשת:

- Constructor
- Getter לכל שדה
- `hashCode()` ו-`equals()`
- `toString()` עבור הלוגים
- לפעמים `Builder`

עבור מחלקה עם 8 שדות, זה בקלות **80 שורות של קוד שחוזר על עצמו**. כל שורה היא מקום שבו מפתח עלול להקליד לא נכון, לשכוח שדה ב- `equals()`, או להעתיק `getter` שגוי.

הפתרון ב-Java 21

Record הוא הצהרה בשורה אחת שנותנת לך אוטומטית את כל מה שלמעלה. הקוד החוזר נכתב על ידי הקומפיילר; המפתח כותב רק את המשמעות.

דוגמה בנקאית

Java 8 – ה- `Transaction` שלנו הוא בערך 60 שורות:

```
public class Transaction {
    private final String id;
    private final String fromAccountId;
    private final String toAccountId;
    private final BigDecimal amount;
    private final String currency;
    private final TransactionType type;

    public Transaction(String id, String from, String to,
                       BigDecimal amount, String currency, TransactionType type) {
        this.id = id;
        this.fromAccountId = from;
        // וכך הלאה לכל שדה...
    }

    public String getId() { return id; }
    public String getFromAccountId() { return fromAccountId; }
    // אחד לכל שדה getter

    @Override
    public boolean equals(Object o) { /* 15 שורות */ }

    @Override
    public int hashCode() { /* 5 שורות */ }

    @Override
    public String toString() { /* 5 שורות */ }
}
```

Java 21 – אותו דבר, בשורה אחת:

```
public record Transaction(
    String id,
    String fromAccountId,
    String toAccountId,
    BigDecimal amount,
    String currency,
    TransactionType type
) {}
```

הקומפיילר יוצר עבורך את ה-constructor, ה-accessors, equals, hashCode ו-toString.

למה לבנק להתעניין

- **פחות באגים.** פחות קוד שנכתב ביד = פחות טעויות באובייקטים שקשורים לכסף.
- **פיתוח מהיר יותר.** "Receipt" או "StatementLine" חדש ייווצר בשניות.
- **קוד-ריוויזי קל יותר.** המבקרים רואים מה ייחודי במחלקה, במקום לעיין ב-60 שורות של boilerplate.

3. Pattern Matching עם Sealed Types – הקומפיילר תופס מקרים שנשכחו

הבעיה ב-Java 8

תוכנת בנקאות מלאה בלוגיקה של "החלט מה לעשות לפי סוג". לדוגמה, הצגת עסקה:

- DEPOSIT ← "כסף נכנס"
- WITHDRAWAL ← "כסף יוצא"
- TRANSFER ← "נשלח לחשבון אחר"
- FEE ← "עמלת בנק"

ב-Java 8 זה נכתב עם משפטי if / else ו-switch. הבעיה הגדולה: **אם מפתח מוסיף סוג עסקה חדש מאוחר יותר** (למשל CHARGEBACK), שום דבר לא מזכיר לו לעדכן כל switch בקוד. הסוג החדש נופל בשקט ל-default, והבנק מציג מידע שגוי ללקוחות.

הפתרון ב-Java 21

Sealed types מאפשרים לנו להגיד לקומפיילר: "אלו הסוגים האפשריים היחידים של Transaction". בשילוב עם **switch pattern-matching**, הקומפיילר עכשיו **מסרב להדר** אם מקרה כלשהו חסר.

דוגמה בנקאית

Java 8 – קל לשכוח מקרה:

```
String describe(Transaction t) {
    if (t.getType() == TransactionType.DEPOSIT) {
        return "Money received: " + t.getAmount();
    } else if (t.getType() == TransactionType.WITHDRAWAL) {
        return "Money withdrawn: " + t.getAmount();
    } else if (t.getType() == TransactionType.TRANSFER) {
        return "Transfer to " + t.getToAccountId();
    }
    // שום דבר לא יזכיר לנו FEE? CHARGEBACK? ומה לגבי
    return "Unknown";
}
```

Java 21 – הקומפיילר מגן עלינו:

```
sealed interface Transaction
    permits Deposit, Withdrawal, Transfer, Fee {}

String describe(Transaction t) {
    return switch (t) {
        case Deposit d    -> "Money received: " + d.amount();
        case Withdrawal w -> "Money withdrawn: " + w.amount();
        case Transfer tr   -> "Transfer to " + tr.toAccountId();
        case Fee f         -> "Bank fee: " + f.reason();
    };
}
```

אם נוסיף סוג Chargeback חדש מחר, הקוד לא יעבור הידור עד שנטפל בו בכל מקום. הבאג נתפס לפני שהוא מגיע ל-production.

למה לבנק להתעניין

- **שינויים בטוחים יותר.** הוספת סוג עסקה חדש, סטטוס חשבון או מוצר לא תוכל בטעות לשבור מסכים קיימים.
- **קוד ברור יותר.** כל המקרים האפשריים מופיעים במקום אחד.
- **פחות בדיקות ידניות.** הקומפיילר עושה את הבדיקה "האם שכחתי משהו?" בחינם.

4. Structured Concurrency – עבודה מקבילה נקייה יותר

הבעיה ב-Java 8

כאשר לקוח פותח את האפליקציה הסלולרית, הבנק לעיתים צריך לטעון במקביל מספר דברים:

- יתרות חשבונות

- עסקאות אחרונות
- כרטיסים פעילים
- הלוואות פתוחות

ב-Java 8 זה נעשה עם `CompletableFuture` והרבה קוד דבק. החלק הקשה ביותר הוא **כשל**: אם “טעינת הלוואות” קורסת, מה קורה לשלוש המשימות האחרות? הן ממשיכות לרוץ ברקע, מבזבזות משאבים, ואולי מחזירות נתונים שהמשתמש לעולם לא יראה. אם המשתמש סוגר את המסך – אותו דבר. ניקוי כראוי הוא קשה, ודליפות קורות.

הפתרון ב-Java 21

Structured Concurrency מתייחסת למשימות מקבילות כאל “צוות” קטן שחי בתוך scope ברור. כללים:

- כל המשימות בצוות מסיימות (או נכשלות) ביחד.
- אם אחת נכשלת, האחרות **מבוטלות אוטומטית**.
- אם ההורה מוותר (לדוגמה, המשתמש עזב את המסך), כל הצוות מבוטל.

דוגמה בנקאית

Java 8 – תיאום ידני, קשה לעשות נכון:

```
CompletableFuture<List<Account>> accounts =
    CompletableFuture.supplyAsync(() -> accountService.findByCustomer(id));
CompletableFuture<List<Transaction>> txs =
    CompletableFuture.supplyAsync(() -> transactionService.history(id));
CompletableFuture<List<Card>> cards =
    CompletableFuture.supplyAsync(() -> cardService.findByCustomer(id));
CompletableFuture<List<Loan>> loans =
    CompletableFuture.supplyAsync(() -> loanService.findByCustomer(id));

CompletableFuture.allOf(accounts, txs, cards, loans).join();
// נכשל, השלושה האחרים ממשיכים לרוץ. צריך לזכור לבטל loans אם
```

Java 21 – נקי, אוטומטי, ללא דליפות:

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {

    var accounts = scope.fork(() -> accountService.findByCustomer(id));
    var txs       = scope.fork(() -> transactionService.history(id));
    var cards     = scope.fork(() -> cardService.findByCustomer(id));
    var loans     = scope.fork(() -> loanService.findByCustomer(id));

    scope.join();           // מחכים לכולם
    scope.throwIfFailed(); // אם מישהו נכשל, האחרים כבר בוטלו

    return new CustomerDashboard(
        accounts.get(), txs.get(), cards.get(), loans.get()
    );
}
```

אם `loans` נכשל, שלוש המשימות האחרות **נעצרות מיידית**. אין דליפות, אין חצי-תוצאות.

למה לבנק להתעניין

- **פחות עומס על השרת.** אין משימות "רפאים" שעושות עבודה שאף אחד לא ישתמש בה.
- **חויית משתמש טובה יותר.** הדאשבורד מופיע מהר יותר כי כל ארבע השאילתות רצות במקביל.
- **קוד בטוח יותר.** ביטול וניקוי מטופלים על ידי השפה, לא על ידי המפתח.

5. SMS – Text Blocks, קבלות ולוגי audit נקיים יותר

הבעיה ב-Java 8

בנק שולח הרבה טקסט מעוצב: **התראות SMS, אישורי email, קבלות מודפסות, רישומי audit log**, **שאילתות SQL**. ב-Java 8, מחרוזות מרובות שורות צריכות להיות מודבקות יחד עם `+` ו-`\n`. התוצאה קשה לקריאה, קל לשבור, והפורמט בקוד לא נראה כמו הפורמט שהלקוח יראה.

הפתרון ב-Java 21

Text Blocks מאפשרים לכתוב מחרוזות מרובות שורות בדיוק כפי שהן צריכות להיראות, בין סימני `"""`. מה שאתה רואה בקוד – זה מה שהלקוח מקבל.

דוגמה בנקאית

Java 8 – קבלה מודפסת:

```
String receipt =
    "=====\n" +
    "      BANK RECEIPT\n" +
    "=====\n" +
    "Account:  " + account.getNumber() + "\n" +
    "Operation: " + type + "\n" +
    "Amount:    " + amount + " " + currency + "\n" +
    "Balance:   " + balance + "\n" +
    "Date:      " + date + "\n" +
    "Reference: " + ref + "\n" +
    "=====\\n";
```

Java 21 – אותה קבלה, קריאה:

```
String receipt = ""
    =====
    BANK RECEIPT
    =====
    Account:    %s
    Operation:  %s
    Amount:     %.2f %s
    Balance:    %.2f
    Date:       %s
    Reference:  %s
    =====
    """".formatted(account.getNumber(), type, amount, currency,
                    balance, date, ref);
```

למה לבנק להתעניין

- **פחות באגי פורמט** בטקסטים פונים-ללקוח (חסר ב-SMS נראה לא מקצועי).
- **קל יותר לעדכן** תבניות כשמרקטינג או compliance מבקשים שינוי ניסוח.
- **SQL נקי יותר** עבור דוחות – שאלות ארוכות נעשות קריאות במקום "מרק של פלוסים".

Modern HTTP Client – אינטגרציה קלה עם שירותים אחרים

הבעיה ב-Java 8

בנק מודרני נדיר חי לבד. הוא צריך לדבר עם:

- **בנק מרכזי** עבור שערי מטבעות
- **שירותי fraud detection**

- **שערי SMS / email**
- **לשכות אשראי** (KYC, credit score)
- **רשתות השלום** (SWIFT, SEPA, רשתות כרטיסי אשראי)

ב-Java 8, כלי ה-HTTP המובנה (`URLConnection`) הוא **מסורבל**: עשרות שורות לבקשה אחת, אין תמיכה בפרוטוקולים מודרניים (HTTP/2), אין `async` קל. רוב הצוותים מוסיפים ספרייה חיצונית (Apache HttpClient, OkHttp), מה שאומר עוד תלויות שיש לנהל ולעדכן.

הפתרון ב-Java 21

ב-Java 21 כבר מובנה **HttpClient מודרני**: סינטקס קצר, תומך ב-HTTP/2 מהקופסה, ועובד מצוין יחד עם Virtual Threads (פרק 1) – כך שאפילו אלפי קריאות חיצוניות מקבילות נשארות זולות.

דוגמה בנקאית

Java 8 – קבלת שער USD/EUR של היום:

```
URL url = new URL("https://api.centralbank.example/rates?base=USD&target=EUR");
URLConnection conn = (URLConnection) url.openConnection();
conn.setRequestMethod("GET");
conn.setConnectTimeout(5000);
conn.setReadTimeout(5000);

try (BufferedReader reader = new BufferedReader(
    new InputStreamReader(conn.getInputStream()))) {
    StringBuilder sb = new StringBuilder();
    String line;
    while ((line = reader.readLine()) != null) sb.append(line);
    String json = sb.toString();
    // ידנית או בעזרת ספרייה אחרת JSON-ואז לפרסר את ה...
}
```

Java 21 – אותה קריאה:

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.centralbank.example/rates?base=USD&target=EUR"))
    .timeout(Duration.ofSeconds(5))
    .build();

String json = client.send(request, BodyHandlers.ofString()).body();
```

שלב את זה עם Records (פרק 2), ופענוח התשובה הוא רק עוד כמה שורות.

למה לבנק להתעניין

- אינטגרציות מהירות יותר. התחברות לשותף fraud או KYC חדש לוקחת שעות במקום ימים.
- פחות ספריות חיצוניות למעקב אחר עדכוני אבטחה.
- ביצועים טובים יותר תחת עומס, הודות ל-HTTP/2 + Virtual Threads.

טבלת סיכום – מה הלקוח מקבל

יכולת	ערך עסקי	תיאור בשורה אחת
Virtual Threads	פחות עלויות חומרה, throughput גבוה יותר	אותו שרת מטפל בפי 10–100 יותר לקוחות
Records	פיתוח מהיר יותר, פחות באגים	60 שורות הופכות ל-1
Sealed + Pattern Matching	שינויים בטוחים יותר	הקומפיילר תופס מקרים שנשכחו
Structured Concurrency	עבודה מקבילה נקייה, ללא דליפות	משימות מקבילות חיות ומתות יחד
Text Blocks	טקסט פונה-ללקוח נקי יותר	מה שכותבים – זה מה שהלקוח רואה
Modern HttpClient	אינטגרציות מהירות ופשוטות יותר	מובנה, ללא ספריות נוספות

סדר מומלץ לדמו

עבור דמו ללקוח אנו ממליצים:

1. **Records** – קל להבנה, אפקט ויזואלי מיידי ("60 שורות ← שורה אחת").
 2. **Virtual Threads** – הטיעון העסקי החזק ביותר, אפשר להראות עם בדיקת עומס.
 3. **Sealed + Pattern Matching** – מסר של איכות ובטיחות; להראות איך הקומפיילר מסרב לבנות כשמקרה חסר.
 4. **Text Blocks + HttpClient** – סיום מהיר של "ויש עוד".
- הסדר הזה הולך מ-קל להבנה ← ביצועים מרשימים ← איכות טובה יותר ← נוחות יומיומית, מה שמשאיר אצל הלקוח רושם כללי חיובי.